Restricting Agents, Guiding Paths, Forming Convoys and Other Mechanisms for Lifelong Multi-Agent Pickup and Delivery in the League of Robot Runners 2026

Jan Wangrat
Politecnico di Milano
jan@wangrat.com

Abstract

The League of Robot Runners is an algorithmic competition in which teams design solutions to Multi-Agent Path Finding (MAPF) and Task Scheduling problems, where a fleet of robots must complete multi-errand tasks that arrive online. This year's edition introduced stochastic execution, which, combined with a delay between scheduling and plan execution, adds uncertainty as a third dimension of the problem. This paper presents the solution of the single-person team Trzy Kwaterki, which placed second in all three competition tracks: Combined, Execution and Scheduling. My scheduler extends the organisers' baseline with task assignment via the Hungarian algorithm for small fleets, and for large ones keeps the baseline's greedy assignment but lets each agent consider only tasks in nearby spatial buckets, so that it completes within the scheduler's time budget. It also adds an activation cap that limits the number of active agents, and specialist agents that only accept tasks whose errands lie in a designated region. My planner builds on the Guided PIBT of Chen et al. (2024), in which PIBT follows congestion-aware guide paths; I compute these guide paths with a batched, multi-threaded version of their traffic-flow optimisation. For the largest competition map, I replace the planner with GeoPIBT, an oracle-guided variant of PIBT that substantially improves throughput. I further modify the heuristic functions: the planner's by adjusting traffic-flow penalties, and those of both the scheduler and the planner by adding one-way cells that restrict travel direction without trapping any agent. Finally, I make the executor less conservative in forming convoys, allowing agents moving in the same direction to follow one another one timestep apart under the competition's geometric collision model. My main finding is that at these fleet sizes the largest gains came from the demand side, controlling how many agents hold tasks and which agents may serve which tasks, rather than from better per-timestep collision resolution.

1 Introduction

Multi-Agent Path Finding (MAPF) is a wide area of Artificial Intelligence and Robotics research that goes back to 1987 (Erdmann and Lozano-Pérez 1987). It covers all the problems where we must design paths for numerous robots (agents) that must reach their goals on a defined map without colliding with each other and as efficiently as possible. It has multiple applications, for example in computer games (Silver 2005), autonomous intersection management (Dresner and Stone 2008), airport surface operations (Morris et al. 2016) and autonomous warehouse operation (Wurman, D'Andrea, and Mountz 2008), which this paper focuses on. There are numerous settings of such a problem, for instance MAPF on general graphs (Stern et al. 2019), in continuous time (Andreychuk et al. 2022) or in three-dimensional space for quadrotor swarms (Hönig et al. 2018). In the League of Robot Runners, the robots operate on a two-dimensional grid.

In MAPF, we are usually looking for a collision-free solution in which every agent reaches its goal in the shortest time. In lifelong MAPF (LMAPF) (Ma et al. 2017), an agent gets a new goal each time it reaches one, and we look for a solution in which the agents reach the most goals in a certain time period. One-shot MAPF is the most classic problem, where a group of agents have their goals pre-assigned, one each. There are numerous solutions to this problem, including prioritised planning (Erdmann and Lozano-Pérez 1987; Silver 2005), Conflict-Based Search (Sharon et al. 2015), LaCAM (Okumura 2023) and PIBT (Okumura et al. 2022), which is widely used in large-scale and lifelong settings, including the winning entries of previous editions of the competition. When the tasks are not pre-assigned, and we can choose which robot does which task, we talk about the Multi-Robot Task Allocation (MRTA) problem (Gerkey and Matarić 2004). Common approaches to it include the Hungarian algorithm (Kuhn 1955), greedy assignment and market-based (auction) methods (Dias et al. 2006). If it is one-shot, but we can do both the assignment and the path finding, we talk about Target Assignment and Path Finding (TAPF) (Ma and Koenig 2016), or Multi-Goal TAPF (MG-TAPF) (Zhong et al. 2022) if an agent has to reach multiple errands to fulfil a task. Finally, Multi-Goal Multi-Agent Pickup and Delivery (MG-MAPD) (Xu et al. 2022), the multi-goal extension of the lifelong Multi-Agent Pickup and Delivery (MAPD) problem (Ma et al. 2017), covers problems in which a task can have any number of goal locations, not only a pickup and a delivery. This is the problem closest to the one we solve in the League of Robot Runners.

In the competition (Chan et al. 2024; League of Robot Runners 2026a), teams are tasked to solve a variant of MG-MAPD on a variety of maps and configurations that span

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Instance	Map	Size	Agents	Timesteps
iron	Iron Harvest	1800×1912	10,000	60,000
orz	orz900d	1491×656	4,000	40,000
bos	Boston	256×256	2,000	10,000
fulfill-A	warehouse	500×140	10,000	30,000
fulfill-B	warehouse	500×140	5,000	20,000
fulfill-C	warehouse	500×140	10,000	25,000
rand-A	random	64×64	1,500	25,000
room-A	room	64×64	200	3,000
room-B	room	64×64	200	10,000
maze-A	maze	32×32	50	5,000
maze-B	maze	32×32	50	4,000

Table 1: Evaluation instances.

different sizes (from about a thousand to millions of cells), numbers of agents (from 50 up to 10,000) and topologies, from very narrow to very open maps. This year's edition introduced stochasticity by adding random delays to agents (as in MAPF with delay probabilities (Ma, Kumar, and Koenig 2017; Zhang et al. 2025)), which makes the problem even harder, as it makes future states hard to predict and requires handling the delays online.

The problem setting introduces different challenges that the mechanisms below try to address. Most importantly, deadlocks must not form, as they are the single biggest issue: a formed deadlock drastically lowers the throughput. Figure 1 shows the largest pile of agents that formed at some point on each map. Each panel shows one map at the moment of its largest pile in my own test runs, with the agents of the pile, those standing side by side, in red; on every map except iron, the pile held at least a third of the fleet. Small maps also introduce different challenges than large ones. On small maps, the time budget is generous, and we can look for near-optimal solutions. On large maps, not just the computation time but even the memory becomes an issue, so contestants need to optimise their implementation in both respects, and every change has to spend the time budget carefully.

2 Problem Setting

An instance consists of a four-connected grid map, a fleet of N agents and an endless stream of tasks; the objective is the number of tasks completed within a fixed number of timesteps (Chan et al. 2024; League of Robot Runners 2026a), and I call the tasks completed per timestep the throughput. The evaluation uses 11 instances on 7 maps (Table 1). The organisers provide a start kit with the simulator and a reference implementation of the scheduler, planner and executor described below, which I also call the baseline. Compared with earlier editions, the 2026 rules add the executor, multi-step plans sent once per round, actions lasting several timesteps, geometric collisions and random delays (League of Robot Runners 2026b).

Tasks. A task is an ordered sequence of two to five errands, each a cell, and is completed when its assigned agent has visited them in order. The scheduler chooses among the revealed tasks that no agent has started; as tasks finish, new ones are revealed, so that the number of revealed, unfinished tasks stays at a fixed value set by each instance. A task is opened when its agent reaches the first errand; before that it may be reassigned, afterwards the same agent must complete it. An agent without a task is free.

Agents. An agent's state is a cell and an orientation. The actions are moving forward, rotating by 90 degrees in either direction, and waiting. A move or rotation takes d timesteps, with $3 \leq d \leq 10$ per instance, and once started can be paused but not aborted. Collisions are geometric: each agent occupies a unit square, two agents collide if their squares overlap during a timestep, and the simulator replaces an unsafe action by a wait. In each timestep, an agent is delayed with a per-instance probability between 0.0005 and 0.02, for 1 to 200 timesteps, during which it waits. A solution observes that an agent is delayed, but not for how long.

Timing. A timestep lasts 100 ms. In each round, which lasts 250 to 5,000 ms depending on the instance, the scheduler assigns tasks and the planner returns a multi-step plan that moves each agent towards its goal, the next errand of its task. The executor stages the plan, and in every timestep it issues GO or STOP to each agent, executing or pausing its next staged action, so it can retime a plan but not change its route. It must answer within the timestep; if it is late, all agents wait. The scheduler and the planner run concurrently with the simulation, so the schedule and the plan take effect only from the next round, and the plan starts from the states the executor predicts for that moment; if the computation is late, agents continue with their staged actions. In the start kit, the scheduler receives half of the round. Instance parameters are hidden, but d and the round length can be read at run time. Each map allows 30 minutes of unscored preprocessing, and evaluation runs on a machine with 32 vCPUs and 64 GiB of memory (League of Robot Runners 2026a).

Tracks and scoring. In the Combined track, a solution replaces all three components and the code that connects them; in the Scheduling and Execution tracks, only the scheduler or the executor, with the start kit's reference components for the rest: a greedy scheduler, the Guided PIBT planner (Chen et al. 2024), and an executor based on a temporal plan graph (Ma, Kumar, and Koenig 2017). Each track has its own leaderboard, so my three entries are three versions of the code (Table 3). On each instance, a submission scores its completed tasks divided by the virtual best, the best result of any submission on that instance, and the track score is the sum over the 11 instances (League of Robot Runners 2026a). Scores therefore fall whenever another submission sets a new best, and teams are ranked by their best single submission. Delays are redrawn in every evaluation, so the same code scores differently each time (Section 7). The evaluation instances are hidden, so my own test runs, marked as farm, use the public maps with my own task files, and their numbers can differ from the evaluation.

3 Scheduler

The scheduler runs once per round and decides which agent works on which task. The start kit's scheduler is greedy:

for each free agent, in no particular order, it scans all free tasks and takes the one with the shortest trip through all its errands, using distance tables built on demand for each goal. It uses at most half of the round, and free agents it has not yet reached by the deadline wait for the next round. It never reconsiders an assignment, and it gives a task to every free agent, regardless of congestion. The following subsections address these points in turn: the quality and cost of the matching (Section 3.1), and how many and which agents work at all (Sections 3.2 and 3.3).

3.1 Task Assignment

Hungarian Algorithm For fleets of up to 200 agents (the room and maze instances), I replace the start-kit greedy assignment with an exact solution of each round's linear assignment problem, using the Hungarian algorithm (Kuhn 1955), as applied to lifelong pickup-and-delivery by Ma et al. (2017). The cost of each agent–task pair is the length of the agent's trip through all of the task's errands. The result is optimal only for the current round, not for the lifelong objective. On larger fleets, building the cost matrix alone exceeds the scheduler's time budget; another team in the previous edition also found exact assignment too slow for large fleets (Yukhnevich and Andreychuk 2025).

Bucketed Greedy Larger fleets keep the greedy assignment, with its scan restricted to spatial buckets: the map is tiled with equal rectangles, each one fifteenth of the map's height and width (rounded down, so 15 or 16 per side), and each agent considers only the tasks whose first errand lies in the rectangles around it, widening the ring until it finds one. On iron, this greedy matching skips an agent whose trip to a task's first errand is longer than a slack τ, unless the revealed tasks outnumber the agents by more than 15%, so agents wait for a closer task instead of crossing the map. The slack is $\tau = 12\sqrt{F/N}$ cells, where F is the number of free cells, that is, 12 times the typical spacing between agents (178 cells on iron). An agent that has been skipped 20 times takes its best task anyway, so no agent waits forever. On orz, the greedy matching also uses a corridor closure: a narrow corridor, at most 5 cells wide, is closed once 6 agents in it have not moved for two rounds, and while it is closed, no task whose shortest route crosses it is assigned, even if the agent stays without a task, until fewer than 3 stuck agents remain.

Rematching The competition allows a task to be reassigned until its agent reaches the first errand; after that, the agent must finish it. The start-kit scheduler never uses this freedom: once an agent holds a task, it is not reconsidered. Task swaps of this kind go back to the token passing with task swaps of Ma et al. (2017), and in the previous edition another team already treated agents with unopened tasks as free (Shen et al. 2025). Rematching reconsiders agents that are still travelling to their first errand. After the normal assignment in each round, two passes run. In both, the cost of a task for an agent is the length of its trip from its current cell through all of the task's errands, as in the assignment.

1. *Agent sweep.* For each agent with an unopened task, the cost of that task is compared with the cheapest free task in nearby buckets. The agent switches if the new task is cheaper by more than $\varepsilon = 4$ cells, a margin for the turns that the cost ignores. The old task returns to the free pool.
2. *New-task steal.* Each newly revealed task goes to a free agent near it, if the activation cap (Section 3.2) allows, or else to the nearby agent that saves the most by swapping its current unopened task for it, provided the saving exceeds ε.

Agents close enough to reach their first errand within the round are skipped, since a change would then be rejected. Agents are only swapped between tasks, never released to idle. To bound its cost, the agent sweep stops after 512 swaps or 200,000 cost evaluations per round, and starts from a different agent in each round, so every agent is reconsidered over several rounds. With the Hungarian algorithm, rematching is simpler: in each round, agents whose tasks are still unopened re-enter the matrix together with their tasks, and the cost of an agent's current task is lowered by ε, so it switches only for a clear gain.

A natural extension for large fleets, which I did not implement, combines the two mechanisms. Because a task stays reassignable until its first errand, an assignment computed on a slightly outdated state remains usable. The Hungarian algorithm could therefore run on large fleets in a background thread, with its result applied lazily by rematching the agents that have not yet reached their first errand.

3.2 Activation Cap

The competition fixes the fleet size of each instance, but it does not require every agent to work. Chen et al. (2024) observe that throughput in lifelong MAPF peaks at some agent density and decreases beyond it, as congestion grows. The activation cap is the most direct way to operate below that peak: at any moment, at most αN of the N agents hold tasks, and the rest stay idle. The fraction α is set per map (Table 2). Three maps run without a cap in the Combined entry: on iron, on bos and on the warehouse. Keeping part of the fleet inactive is not new: earlier competition entries disabled agents or kept only some of them active (Jiang et al. 2024; Yukhnevich and Andreychuk 2025), and Onn, Felner, and Stern (2026) study throughput with unassigned agents.

Reducing the number of working agents helps in four ways:

1. *Fewer deadlocks.* Fewer agents enter the narrow corridors where deadlocks form (Figure 1).
2. *Less interference.* Away from bottlenecks, a sparser fleet obstructs itself less, so active agents move closer to their free-flow speed.
3. *Shorter trips.* When every agent holds a task, even an assignment that minimises the round's total cost must give the last agents tasks far from them, often across half the map. Under the cap, such agents wait until a task appears near them.
4. *Less computation.* As a minor side effect, fewer active agents should also mean fewer guide paths to compute and a smaller assignment problem in each round.



iron t = 54,000 1,121 agents (11%) | **orz** t = 36,000 1,362 agents (34%) | **fulfill-A** t = 27,000 5,868 agents (59%) 3,090 of them in view | **bos** t = 9,999 1,266 agents (63%) | **rand-A** t = 15,000 1,230 agents (82%) | **room-B** t = 7,000 192 agents (96%) | **maze-B** t = 3,600 44 agents (88%)

Figure 1: Largest pile of agents (red) per map.

Map	Combined	Scheduling
orz	0.30	0.25
maze-A, maze-B	0.75	0.75
rand-A	0.75	0.75
room-A, room-B	0.55	0.60
fulfill-B	–	0.80
fulfill-C	–	0.60
iron, bos, fulfill-A	–	–

Table 2: Activation cap α per map; – means no cap.

Two properties keep the cost of idle agents low:

1. *Cheap to keep.* Idle agents cost almost nothing where they stand, because the planner pushes them out of the way of active agents. Leaving them in place worked better than parking them in dedicated locations in some initial tests, since they stay spread across the map.
2. *Reversible.* The cap limits how many agents are active, not which ones. An agent left idle by the cap can receive a task later, so no part of the fleet is permanently discarded.

The mechanism has clear limitations. The fraction α is static and was chosen per map empirically; a cap that adapts to observed congestion is the natural next step. Because evaluation scores are noisy (Section 7), neighbouring values of α often cannot be told apart, so Table 2 lists values that worked, not proven optima. The values are also specific to my planner: Chen et al. (2024) show that better guidance moves the throughput peak to larger fleets. Every planner has its own cap, and a strong enough one might need no cap at all on these maps.

The underlying idea, that the system performs better when it does not use all of its capacity, recurs elsewhere in the scheduler. The specialist agents (Section 3.3) apply it per region instead of globally, the matching slack τ (Section 3.1) targets the trip-length effect in the matching, and on orz the corridor closure (Section 3.1) leaves agents without a task rather than sending them into a jammed corridor.

3.3 Specialist Agents

The first effect of the activation cap, fewer deadlocks (Section 3.2), comes from limiting how many agents are in narrow corridors. A global cap achieves this only by thinning the whole fleet, including open areas that have no bottleneck,

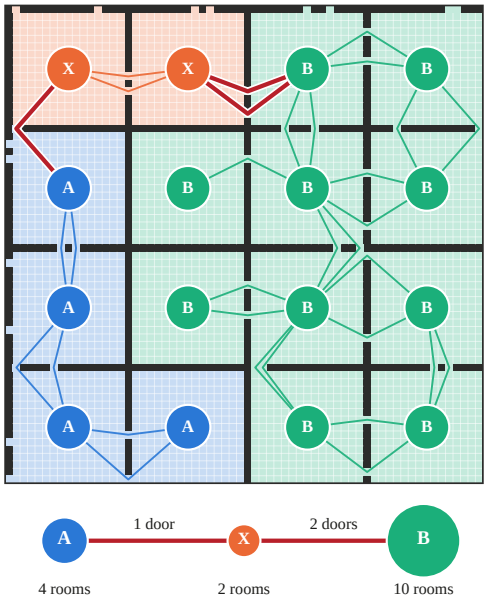
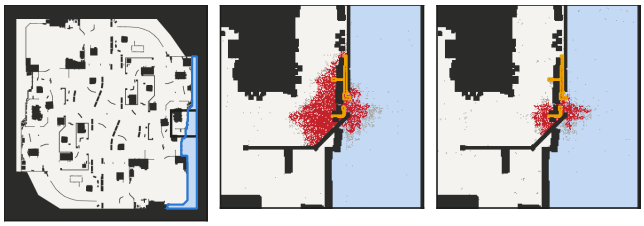


Figure 2: Room map: zones and room graph.

where the cap might only costs throughput. With specialist agents, the fleet is split into groups, and each agent accepts only tasks whose errands lie in its group's region. The size of the group whose tasks cross a bottleneck then acts as an activation cap on that bottleneck alone, while agents in open regions work without restriction. The regions are drawn by hand; the group sizes are either estimated from how the first revealed tasks split between the regions, as on room, or set by hand from intuition and evaluation results, as on iron. Confining agents to zones appears in multi-agent pickup and delivery with agents restricted to operable zones (Flammini, Amigoni, and Lacerda 2026).

Figure 2 shows the room map, where two open zones, A and B, are connected only through a bottleneck zone X. To illustrate the idea, consider a simple model in which each task has two errands placed uniformly at random, and A, X and B contain a quarter, an eighth and five eighths of the errand locations (4, 2 and 10 of the 16 rooms). Tasks then fall into three types:

1. both errands in A: $1/16 \approx 6\%$ of tasks;



(a) iron map
blue: pocket
box: area of (b), (c)

(b) no specialist mechanism
t = 59,999
2,865 at the entrances

(c) 5% specialists
t = 59,999
901 at the entrances

Figure 3: Iron pocket (blue) and its entrances (yellow).

2. both errands in B: $25/64 \approx 39\%$;
3. all remaining tasks, which touch or cross X: $\approx 55\%$.

With more than two errands per task, the share of tasks that stay within A or B shrinks quickly. At run time, the shares are estimated from the first tasks revealed on each instance, and the fleet is split into three matching groups: specialists for A, specialists for B, and a crossing group that serves the third type. On my 200-agent test instance, these groups had 11, 79 and 110 agents, and part of the crossing group can then be kept idle on top of this split, as described below. Since tasks within A and B are shorter than crossing tasks, shares of tasks overestimate the agent time A and B need and underestimate that of X. This bias is desirable. A and B have no bottleneck, so surplus agents there carry little risk and keep their throughput at its maximum, and in my test runs the specialists kept A and B served at all times. The crossing group, in turn, had to be reduced by an activation cap anyway. The split brings three benefits:

1. *Bounded exposure.* Because tasks arrive at random, demand for crossing tasks fluctuates and at times exceeds what the bottleneck can carry. With specialists, no more agents can be crossing at once than the crossing group contains, and the excess tasks wait until a crossing agent becomes free.
2. *Isolation.* Specialists for A and B never enter X, so if X jams, they keep completing tasks. In the model above, they make up about 45% of the fleet, so a jam lowers throughput but can only stop it after a long time, when all tasks are of type X.
3. *Per-group caps.* Each group can receive its own activation cap, strict for the crossing group and loose or absent in the open zones.

On room-A and room-B, the group sizes follow the shares estimated at run time. In the Combined entry, no group has its own cap: the crossing group is limited by the bridge semaphore (Section 4.5) and the whole fleet by a cap of $0.55N$. The Scheduling entry, which cannot use semaphores, gives tasks to only 33 agents of the crossing group and keeps the rest of it idle, under a cap of $0.60N$.

On iron, the largest map, a pocket region (Figure 3a) that received 6.77% of the tasks in my test runs is served, in the Combined entry, by a reserved group of 7% of the fleet. In a 60,000-timestep test run, a group of 5% already cut the number of agents at the pocket entrances from 2,865 to 901 (Figure 3b,c). In my test runs, reserving more than the pocket's demand share caused a sharp loss rather than a gradual one: 15% of the fleet completed 19% fewer tasks than 7%.

The main drawback is the flip side of bounded exposure: a fixed split also stops an open zone from drawing extra agents when its demand spikes, which, unlike at the bottleneck, is a pure loss. Over a longer run, the split can also lead to starvation: if most tasks come to lie in one group's region, they wait for that group's agents while the agents of the other groups stand idle. The split also assumes representative early tasks and hand-drawn zones. It should not be worse than no specialists though in that manner since correctely selected cap is actually making the throughput better on all the areas.

4 Planner

The planner turns the agents' current goals into a plan of several steps, starting from the states the executor predicts. The start kit's planner is the Guided PIBT of Chen et al. (2024). In each round, it computes a guide path to the goal for every agent whose goal has changed, improves the guide paths by Frank–Wolfe traffic-flow optimisation until its time budget runs out, and then moves the agents along them over the planning window with Priority Inheritance with Backtracking (PIBT) (Okumura et al. 2022), in which agents pick their next cell one at a time in priority order and may push lower-priority agents aside. It runs on a single thread, although the evaluation machine has 32 vCPUs. I use three variants, chosen by map size and fleet: Guided PIBT with parallel guidance for fleets of 2,000 agents or more (Section 4.1), the sequential version on smaller fleets, with a trial ensemble on room and maze, and GeoPIBT on iron (Section 4.3). The remaining subsections change what the planner works with: the cost of moves (Section 4.2), one-way cells (Section 4.4) and semaphores (Section 4.5).

4.1 Guided PIBT

Parallel Frank–Wolfe Guidance On every map except iron, my planner is a fork of the start-kit planner. Here I keep its PIBT core unchanged and parallelise the computation of the guide paths; Section 4.2 changes their cost.

In each Frank–Wolfe step, the start kit re-plans the guide paths one agent at a time, each against the flow left by the previous ones. In my version, a batch of agents, eight per thread, re-plans in parallel against a frozen snapshot of the flow, and the new paths are then committed one by one. The same batching is used to re-plan agents whose goal has changed, which the start kit also does one by one and which was the main serial bottleneck in profiling. Agents that share a goal are placed in different batches, and the per-goal heuristic tables are built before the parallel step, to avoid races. Parallel Frank–Wolfe with delayed, batched updates is known in optimisation (Wang et al. 2016); what is specific here is its use for guide paths under a per-round deadline. On the farm, this made 1.53 times as many path updates per round as the sequential version with 4 threads, and 1.66 times with 8; the serial commit limits the gain.

The drawback is that agents within a batch do not see each other's new paths. On small fleets this loses much of the coordination of the sequential version: on the farm, the parallel

version lowered throughput by 32% on room and by 7.5% on maze. I therefore use it only for fleets of 2,000 agents or more, that is, on orz, bos and fulfill-A, B and C, while rand-A, room and maze keep the sequential version. In the evaluation, the number of completed tasks rose by 20% on fulfill-A and fulfill-B and by 12% on bos, comparing the submissions just before and just after the change; fulfill-C stayed within noise. On orz, the change arrived together with the activation cap, so its effect cannot be separated there; on the farm, it completed 22–37% more tasks.

I also tried a hybrid in which the parallel version runs for 85% of the time budget and a sequential pass polishes the result in the remaining 15%. It did not improve the score, and I abandoned it, most likely because the sequential pass took time from the parallel iterations, which mattered more than the staleness it removed, although I still believe the direction is right and that the problem may have been in my implementation.

Further research could try better heuristics for choosing batches of agents whose goals do not overlap and who would not interfere, as well as modelling how the number of path updates per round and the reduction in the sum of costs scale with the number of threads.

Small-Map Trial Ensemble PIBT resolves conflicts greedily in priority order, so the order affects which agents make progress (Okumura et al. 2022). On small maps, the guidance has few agents to update, so I take part of its time budget to search over priority orders. After computing the Frank–Wolfe guides, I run K trials of PIBT for $H = 12$ steps from the same state, with $K = 16$ on maze and $K = 8$ on room. The trials cycle through four priority orders, so trials with the same order differ only in random tie-breaks. The start-kit order puts first the agents that have pursued their current goal the longest, moves agents out of dead ends early, and puts agents without a task last, breaking ties randomly. The other three orders are closest-to-goal first, farthest-from-goal first, and the start-kit order with random noise. I commit the first steps of the trial whose agents end closest to their errands, measured by the summed guide distance. Since the first trial is the unchanged start-kit rollout, the chosen trial is never worse by this measure. Picking the best of several PIBT runs is not new: Engineering LaCAM* keeps the best of k single-step PIBT runs (Okumura 2024), and WPPL runs PIBT over a window and executes only its first steps (Jiang et al. 2024). My version differs in the 12-step lifelong rollout and the explicit choice of priority orders.

On the farm, the ensemble raised throughput by 8–9% on maze and room. In the evaluation, it gave no visible signal on room, but improved the score by about 6% on maze-A and about 21% on maze-B.

On larger fleets the ensemble lowered throughput, partly because the trials, which run one after another, take time away from the Frank–Wolfe guidance, so it runs only on maze and room.

4.2 Traffic-Flow Penalties and Jam Detour

The guide-path search of the start kit gives each move from cell u to a neighbouring cell v the cost

$$c(u, v) = 1 + (f_{uv} + 1)\, f_{vu} + \lfloor f_v/2 \rfloor,$$

where f_{uv} is the number of guide paths that use the move from u to v, f_{vu} the number that use it in the opposite direction, and f_v the number of guide paths that pass through v.

In my planner, the cost of a move becomes

$$c(u, v) = 1 + \omega\,(f_{uv} + 1)\, f_{vu} + \lfloor f_v/2 \rfloor + \beta\, t_{uv} + \gamma\, w_v + \delta\, j_v.$$

With $\omega = 1$ and $\beta = \gamma = \delta = 0$ it reduces to the start kit's cost; all weights are set per map. The four weighted terms are:

- *Opposing flow* (ω). A higher weight makes moving against planned traffic more expensive, so trips in opposite directions take different aisles and one-way lanes emerge without hard rules. Shipped with $\omega = 6$ on fulfill-A and fulfill-C.
- *Turns* (β). t_{uv} is the number of quarter turns needed to enter v, two for a U-turn. A quarter turn takes as long as a move, but the start-kit guide paths ignore turns and zig-zag; charging for them gives straighter paths. Shipped with $\beta = 1$ on orz, rand-A and the three fulfill instances; on the other maps it made no difference. I believe turns should in general be part of the cost, but I could not show a gain at evaluation.
- *Delay field* (γ). w_v is a fading count of rounds in which agents stood still in v: each such round adds one, and the field decays by 1/32 per round, so a cell blocked in every round settles at 32. The rate of 1/32 is my choice; in my test runs on bos, a slower decay of 1/64 scored the same. The flow terms only see planned traffic, while this term remembers where queues keep forming. Shipped with $\gamma = 3$ on orz and bos. At evaluation it gained about 5% on bos, within noise, and nothing visible on orz.
- *Jam detour* (δ). The map is divided into tiles of 16×16 cells, and $j_v = \lfloor (s_v - 8)/8 \rfloor$ when the tile of v holds $s_v > 8$ agents that did not move since the last round, and 0 otherwise; the tile of the agent's own goal is exempt. Jams form as blobs of stopped agents that the flow terms cannot see, and counting only stopped agents leaves busy but flowing areas untouched. Shipped with $\delta = 4$ on fulfill-A and fulfill-C. The corridor closure (Section 3.1) is its counterpart in the scheduler.

4.3 GeoPIBT

The iron map has 3.4 million cells, 2.2 million of them free. On a map of this size, exact distances to every goal are too expensive: a full cell-to-cell distance table would take about 10 TB. For iron only, I therefore replace the planner with GeoPIBT, a PIBT-style planner (Okumura et al. 2022) built around a coarse distance oracle.

The map is cut into 8×8 blocks, and each block is split into its connected free pieces, which become the nodes of a graph; two nodes are linked when their cells touch. Iron has

37,756 nodes, so the table of hop distances between all pairs of nodes fits in 2.85 GB. The distance from a cell c to a goal g is estimated as

$$h(c, g) = \max\big(\|c - g\|_1,\ 8\,(m - 1)\big) + s,$$

where m is the number of hops between their nodes. In open terrain the Manhattan term dominates and gives a smooth gradient; around obstacles the hop term takes over and accounts for the detour. Because the hop term is flat inside a block, agents initially had no direction there and stalled in a regular pattern, so a small extra term s, always smaller than one hop and zero where the Manhattan term dominates, pulls each agent towards the border of the next node on its way. The one-way cells (Section 4.4) are built into this graph. Splitting a grid into blocks and their connected pieces follows the abstractions used in single-agent pathfinding (Botea, Müller, and Schaeffer 2004), and in the 2023 edition, Puzzle Heuristics also kept only distances between areas around pivot cells to fit the memory limit (Lee, Sim, and Nam 2024). My version adds the combination of hop counts with the Manhattan distance, the pull towards the next border, and the one-way cells.

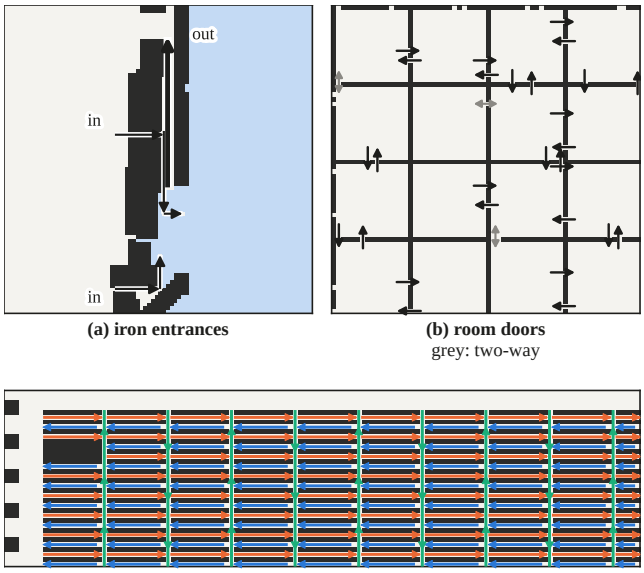
GeoPIBT does not build a path. In each step, agents act in priority order and move to the free neighbouring cell that best decreases h, with a cost for turning and a bonus for keeping the current heading. Priorities persist across rounds, as in PIBT, but there is no priority inheritance or backtracking: a blocked agent asks the lower-priority agent in its way to step aside and waits one step. As a result, routes emerge as a descent on h, and agents heading the same way form convoys. Since the result depends on tie-breaking, 32 rollouts with different tie-breaks run in parallel, and the one with the largest total decrease of h is kept. Agents that stay pinned for many rounds also get a short local detour from a Dijkstra search around the obstacle or jam.

At evaluation, first GeoPIBT submission scored 32,648 tasks on iron, against 16,763 for my guided planner, almost twice as many.

4.4 One-Way Cells

A simple mechanism that proved effective in previous editions is to add very high penalties for moving through certain cells in the wrong direction, which makes movement in some lanes one-way (Jiang et al. 2024; Yukhnevich and Andreychuk 2025). In my solution, the wrong-way moves are forbidden outright rather than penalised, as in the one-way grids of Wang and Botea (2008) and of a 2023 entry (Ambrus-Dobai 2024); unlike their map-wide directions, the cells I use cover only a few lanes. They are built into the distance oracle (Section 4.3), which the scheduler also uses, so both task assignment and movement respect them. I mark the cells by hand: an automatic method is hard to design, whereas deciding how traffic on a map should flow is intuitive for a human. I tried three forms of one-way cells (Figure 4), and only the first was shipped:

1. two entrance lanes and one exit lane through a bottleneck on the iron map, marked cell by cell;
2. one-way doors, each a single cell, on the room map;



(a) iron entrances **(b) room doors**
grey: two-way

(c) warehouse (fulfill), one-way lanes along the blocks
orange: east, blue: west, green: north/south

Figure 4: One-way cells on iron, room and warehouse.

3. parallel one-way lanes on the warehouse map.

On iron, the mechanism proved very effective: it almost eliminated the congestion at the bottleneck, and on a map of this size the cost of a longer route through that small area is negligible. In my own 60,000-timestep test runs, the one-way lanes cut the number of agents at the pocket entrances from 2,865 (Figure 3b) to 22 and raised throughput by 28%.

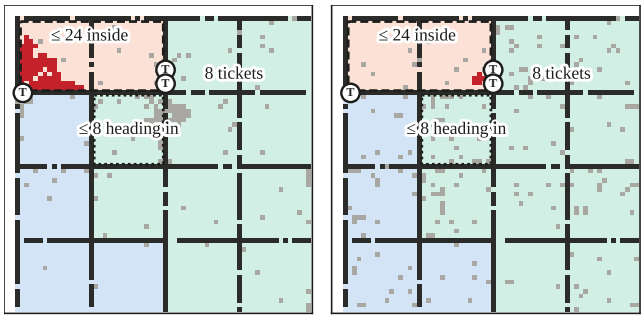
The one-way doors on room did not give a clear signal, and I did not ship them.

On the warehouse map, the lanes alternate east and west from one aisle to the next, and the vertical seams between shelf blocks alternate north and south (Figure 4c). The shipped configuration uses a soft opposing-flow penalty instead (Section 4.2).

On orz, I tested one-way cells only in the narrow necks of the corridors and rejected them. In hindsight, this was likely a mistake: the winning team directed the whole road network by hand, marking thousands of cells, and scored about 2.6 times as many tasks on this map as my scored entry, although their solution also differs in other components.

4.5 Semaphores

The specialist agents (Section 3.3) limit traffic through the bottleneck only indirectly, through a fixed split of the fleet. Semaphores limit it directly. A similar traffic light, admitting a single train into a cluster at a time, was the most effective component of a Flatland competition solution (Laurent et al. 2021). A semaphore holds a fixed number of tickets, and an agent needs a ticket to head into, or enter, a gated area. In each round, after the scheduler has assigned tasks and before the planner runs, every agent's next errand is checked against the semaphores. An agent that is denied keeps its task, but its goal is cleared for the round, so it waits in place or is



(a) zones only
t = 4,000
87 agents in the corner rooms
room 5: 30 inside, 22 at its door

(b) zones + semaphores
t = 4,000
18 agents in the corner rooms
room 5: 24 inside, 1 at its door

Figure 5: Semaphore gates on room-B: corner rooms (dashed), room 5 (dotted), crossing tickets (T).

pushed aside by the planner, and it is checked again in the next round.

I use three semaphores on the room map (Figures 2 and 5):

1. *Bridge.* Eight crossing tickets for the crossing group. An agent of this group needs a ticket only when it stands in A or B and its next errand is on the other side, and it returns the ticket once it arrives there. Errands on the same side need no ticket.
2. *Room.* At most eight agents may have their next errand in room 5, a dead-end room of zone B just below the corner rooms, where the jam re-formed once the corner was limited.
3. *Region.* At most 24 agents may be inside the two corner rooms, 0 and 1. Only agents within one room of the region whose route passes through it are held, because holding agents further away led to gridlock; an agent is held as soon as it enters the neighbouring room, so it waits there and not in the doorway, where it would block the door. Agents already inside are never held, so the region can always empty.

I set all three values by hand from test runs on room-B; a region cap of 18 instead of 24 gridlocked the map. The first two semaphores depend on where an agent is heading, and the third on where agents are, so it also catches agents passing through. In Figure 5, the semaphores keep the corner rooms clear and remove the queue at the door of room 5. Because they act on the planner's input, the semaphores run only in the Combined entry.

5 Executor

The executor is new in this edition: the planner decides where agents go, and the executor decides when, by issuing GO or STOP to every agent in every timestep (Section 2). The start kit's executor is based on a temporal plan graph (Ma, Kumar, and Koenig 2017). It records, for every cell, the order in which agents are planned to visit it, and lets an agent enter a cell only after all earlier visitors have completely left it; rotations are always allowed. After each new plan, it simulates the staged actions without delays, removes those that would not finish, and reports the resulting states as the predicted states for the next round.

5.1 GeoExec

My largest single gain comes from a simple observation. The start-kit rule is safe but slow, since every cell handoff costs two full action durations. Under the simulator's collision rule, however, two agents moving in the same direction at the same speed never overlap, so a follower can start moving one timestep after its leader. GeoExec decides GO or STOP in every timestep with the simulator's own geometry: an agent may move only if its swept square hits no standing agent and no agent already allowed to move in that timestep. It keeps the start kit's order of visits to each cell, and its prediction step simulates the new rule instead of the old one. Since no GO relies on another agent actually moving, a random delay can stop any agent without causing a collision. Su, Veerapaneni, and Li (2024) also let an agent follow one step behind another in a temporal plan graph, but in unit time and relying on an onboard sensor that stops the follower when its leader stops; in GeoExec, safety comes from the GO rule itself.

The difference grows with the length of a convoy. For a convoy of n agents moving ten cells, the last agent arrives after $(n-1)\,d + 10d$ timesteps with the start-kit rule, but after $(n-1) + 10d$ with GeoExec; for $n = 10$ and $d = 3$, that is 57 versus 39.

On the Execution track, the first GeoExec entries completed 7–87% more tasks than my earlier entries on nine of the 11 instances, with no change on iron. On fulfill-A, however, they completed 46% fewer, a loss that persisted in all later evaluations and that I could not explain, so the Execution entry uses the start-kit executor there.

5.2 Delay-Anticipating Holds

The prediction step of GeoExec simulates each new plan without delays, so it assumes that an agent delayed now will move again in the next timestep. Zhang et al. (2025) likewise obtain the next plan's start states by simulating the current plan assuming no additional delays. Since a delay can last up to 200 timesteps, the planner then keeps sending other agents into cells that the delayed agent will block for much longer.

A solution sees that an agent is delayed but not for how long (Section 2), so I simply assume a fixed length: in the simulation, every currently delayed agent is held still for the first k timesteps. Its own remaining actions, and the moves of the agents queued behind it, then no longer finish within the simulated round and are removed from their queues. The predicted states therefore show the delayed agent where it stands and its followers stopped behind it, and the next plan starts from these positions instead of from progress that the delay prevents. The GO/STOP rule stays the same, but an agent whose delay ends before k timesteps has no actions left and waits for the next plan.

On rand-A the holds lost throughput (-8.6% at evaluation), so in the Combined entry they run only on iron and the three warehouse instances: $k = 5$ on the warehouse and $k = 40$ on iron, where evaluation scores still rose with k up

	iron	orz	bos	fulfill			rand	room		maze	
				A	B	C	A	A	B	A	B
Combined entry: scheduler											
Hungarian assignment								•	•	•	•
Matching slack τ	•										
Corridor closure		•									
Rematching		•									
Activation cap α		0.30					0.75	0.55	0.55	0.75	0.75
Specialist agents	•							•	•		
Combined entry: planner											
Guided PIBT		•	•	•	•	•	•	•	•	•	•
Parallel Frank–Wolfe guidance		•	•	•	•	•					
Trial ensemble K								8	8	16	16
Opposing-flow penalty				6		6					
Turn penalty		•		•	•	•	•				
Delay field		•	•								
Jam detour				•		•					
GeoPIBT	•										
One-way cells	•										
Semaphores								•	•		
Combined entry: executor											
GeoExec	•	•	•	•	•	•	•	•	•	•	•
Delay holds k	40			5	5	5					
Scheduling entry											
Hungarian assignment								•	•	•	•
Corridor closure		•									
Rematching		•							•		
Activation cap α		0.25			0.80	0.60	0.75	0.60	0.60	0.75	0.75
Specialist agents	•							•	•		
Execution entry											
GeoExec	•	•	•		•	•	•	•	•	•	•
Reference executor				•							
Delay holds k						5	5				

Table 3: Mechanisms per instance in the three scored entries.

Track	Place	Team	Score
Combined	1	No Man's Sky	10.63
	2	Trzy Kwaterki	7.03
	3	Ash	6.50
Execution	1	mygo	4.71
	2	Trzy Kwaterki	4.61
	3	Turbo Babaa	4.20
Scheduling	1	No Man's Sky	5.08
	2	Trzy Kwaterki	4.98
	3	hcp	4.92

Table 4: Final standings.

to that value. A fixed k is the crudest possible estimate. I believe any solution should account for delays in some form, and a better one would estimate the remaining delay from its distribution.

6 Results

I report the final standings (Table 4), the mechanisms per instance in the three scored entries (Table 3) and the estimated gain of each mechanism in the Combined score (Table 5).

7 Development Workflow

Until mid-June, I developed on a laptop, which could run only small and short experiments, so I relied mostly on the evaluation results. To run full-length simulations, I rented 16-vCPU machine for a month. This is when my score started to rise again: for the first time, I could read detailed logs of the mechanisms I implemented and study how traffic developed on each map. These are the farm runs reported throughout the paper.

Farm results did not always carry over to the evaluation. The evaluation instances are hidden, so my task files and delay settings only approximated them, and the planner's time budget is wall-clock, so the speed of the machine matters. On fulfill-A, one change raised throughput by 49% on the farm and by 0.3% at evaluation. The evaluation is also noisy: delays are redrawn every time, and over 26 submissions with unchanged settings, the middle half of the scores spread by 2% on iron and up to 13% on orz, so a change of a single submission below about 4% on iron, or 15% on orz, could not be told from chance. I therefore tested changes that add computation also with half of the time budget, and trusted an improvement only once two or more submissions agreed.

To make these logs usable, I built a small website that collected all my experiments, with charts of each run's score against the best known result, of where agent time went

Mechanism	Score
Scheduler	
Hungarian assignment	+0.20
Bucketed greedy	–
Matching slack τ	+0.01
Corridor closure	+0.02
Rematching	+0.03
Activation cap α	+0.50
Specialist agents	+0.23
Planner	
Parallel Frank–Wolfe guidance	+0.26
Trial ensemble K	+0.17
Opposing-flow penalty	+0.12
Turn penalty	+0.05
Delay field	+0.03
Jam detour	+0.27
GeoPIBT	+0.31
One-way cells	+0.14
Semaphores	+0.22
Executor	
GeoExec	+0.96
Delay holds k	+0.09

Table 5: Estimated gain of each mechanism in the Combined score.

(moving, waiting, blocked or idle) and of congestion per cell, my best result on each map, and a visualiser of the runs (Figure 6).

I worked with Claude Code, which wrote all of the code. This let me test far more ideas than I could have implemented by hand, but it also introduced many mistakes, some of which I noticed only much later. For example, for about a week my farm runs silently ran without the per-map settings, so most of my mechanisms were off and the baseline I compared against was not my real one. In my experience, giving the agent tools to inspect the details of each run is essential: without them, its analyses were often wrong.

To test hypotheses, I often used a workflow similar to Karpathy's autoresearch[1]: I left Claude running overnight on the rented machine, with instructions to implement an idea, test it first on short runs, and run it at full length only if the short runs gave a positive signal.

8 Conclusion

Many of my biggest wins came from hard gates keeping agents from acting freely: the activation cap, the specialist agents, the semaphores and the corridor closure. Each of them answers the same question in its own way: does one more working agent add more throughput than it takes away through congestion? Probably a more robust solution could give every possible assignment a score, and leave the agent idle whenever the score is below zero.

Barely any of my mechanisms are robust. They are tuned, or drawn by hand, for their instances, and many of them help on one map and hurt on another. Future work could model

[1]https://github.com/karpathy/autoresearch

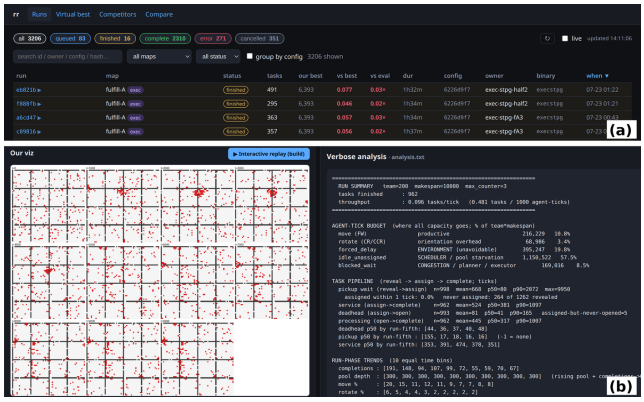


Figure 6: Experiment website: (a) list of runs, (b) one run with its frames and statistics.

them so that they work on any map, switching on only when and where they are needed.

Three things helped me the most in development (Section 7). A strong machine to run my own evaluations is not optional; I believe it is a winning condition, and my score started to rise again only once I had one. My own website and workflow were extremely helpful for making sense of the logs. Finally, LLMs are very helpful for running many small experiments that give signals about interesting mechanisms, and for a first test of new ideas, as long as they have tools to inspect the runs.

References

Ambrus-Dobai, M. 2024. Description of Ideas Used in League of Robot Runners 2023. League of Robot Runners 2023 team report, https://expo24.leagueofrobotrunners.org/resources/M%C3%A1rton_Ambrus-Dobai.pdf.

Andreychuk, A.; et al. 2022. Multi-Agent Pathfinding with Continuous Time. *Artificial Intelligence*, 305: 103662.

Botea, A.; Müller, M.; and Schaeffer, J. 2004. Near Optimal Hierarchical Path-Finding. *Journal of Game Development*, 1: 7–28.

Chan, S.-H.; et al. 2024. The League of Robot Runners Competition: Goals, Designs, and Implementation. In *ICAPS 2024 System Demonstrations Track*.

Chen, Z.; et al. 2024. Traffic Flow Optimisation for Lifelong Multi-Agent Path Finding. In *AAAI*, 20674–20682.

Dias, M. B.; et al. 2006. Market-Based Multirobot Coordination: A Survey and Analysis. *Proceedings of the IEEE*, 94(7): 1257–1270.

Dresner, K.; and Stone, P. 2008. A Multiagent Approach to Autonomous Intersection Management. *Journal of Artificial Intelligence Research*, 31: 591–656.

Erdmann, M.; and Lozano-Pérez, T. 1987. On Multiple Moving Objects. *Algorithmica*, 2(1–4): 477–521.

Flammini, B.; Amigoni, F.; and Lacerda, B. 2026. Multi-Agent Pickup and Delivery with Heterogeneous Agents. In *AAMAS*.

Gerkey, B. P.; and Matarić, M. J. 2004. A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems. *International Journal of Robotics Research*, 23(9): 939–954.

Hönig, W.; et al. 2018. Trajectory Planning for Quadrotor Swarms. *IEEE Transactions on Robotics*, 34(4): 856–869.

Jiang, H.; et al. 2024. Scaling Lifelong Multi-Agent Path Finding to More Realistic Settings: Research Challenges and Opportunities. In *SoCS*.

Kuhn, H. W. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2): 83–97.

Laurent, F.; et al. 2021. Flatland Competition 2020: MAPF and MARL for Efficient Train Coordination on a Grid World. In *NeurIPS 2020 Competition and Demonstration Track, PMLR 133*, 275–301.

League of Robot Runners. 2026a. Problem Setup, Evaluation and Rules. https://www.leagueofrobotrunners.org.

League of Robot Runners. 2026b. Start Kit, Version 3.1.0. https://github.com/MAPF-Competition/Start-Kit.

Lee, W.; Sim, J.; and Nam, C. 2024. Puzzle Heuristics: Efficient Lifelong Multi-Agent Pathfinding Algorithm for Large-scale Challenging Environments. League of Robot Runners 2023 team report, https://expo24.leagueofrobotrunners.org/resources/AIRLAB.pdf.

Ma, H.; and Koenig, S. 2016. Optimal Target Assignment and Path Finding for Teams of Agents. In *AAMAS*, 1144–1152.

Ma, H.; Kumar, T. K. S.; and Koenig, S. 2017. Multi-Agent Path Finding with Delay Probabilities. In *AAAI*, 3605–3612.

Ma, H.; et al. 2017. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks. In *AAMAS*, 837–845.

Morris, R.; et al. 2016. Planning, Scheduling and Monitoring for Airport Surface Operations. In *AAAI-16 Workshop on Planning for Hybrid Systems*.

Okumura, K. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *AAAI*, 11655–11662.

Okumura, K. 2024. Engineering LaCAM*: Towards Real-Time, Large-Scale, and Near-Optimal Multi-Agent Pathfinding. In *AAMAS*, 1501–1509.

Okumura, K.; et al. 2022. Priority Inheritance with Backtracking for Iterative Multi-Agent Path Finding. *Artificial Intelligence*, 310: 103752.

Onn, O.; Felner, A.; and Stern, R. 2026. Maximizing Throughput in Lifelong Multi-Agent Path Finding with Unassigned Agents. In *SoCS*, 121–128.

Sharon, G.; et al. 2015. Conflict-Based Search for Optimal Multi-Agent Pathfinding. *Artificial Intelligence*, 219: 40–66.

Shen, Q.; et al. 2025. Scalable Online Multi-Agent Path Planning: A Hybrid Method with Adaptive Scheduling and Conflict-Minimizing Conflict-Based Search. League of Robot Runners 2024 team report, https://expo25.leagueofrobotrunners.org/resources/Ausdroid-unimelb.pdf.

Silver, D. 2005. Cooperative Pathfinding. In *AIIDE*, 117–122.

Stern, R.; et al. 2019. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *SoCS*, 151–158.

Su, Y.; Veerapaneni, R.; and Li, J. 2024. Bidirectional Temporal Plan Graph: Enabling Switchable Passing Orders for More Efficient Multi-Agent Path Finding Plan Execution. In *AAAI*, 17559–17566.

Wang, K.-H. C.; and Botea, A. 2008. Fast and Memory-Efficient Multi-Agent Pathfinding. In *ICAPS*, 380–387.

Wang, Y.-X.; et al. 2016. Parallel and Distributed Block-Coordinate Frank-Wolfe Algorithms. In *ICML*, PMLR, 1548–1557.

Wurman, P. R.; D'Andrea, R.; and Mountz, M. 2008. Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses. *AI Magazine*, 29(1): 9–19.

Xu, Q.; et al. 2022. Multi-Goal Multi-Agent Pickup and Delivery. In *IROS*, 9964–9971.

Yukhnevich, E.; and Andreychuk, A. 2025. Enhancing PIBT via Multi-Action Operations. League of Robot Runners 2024 team report, https://expo25.leagueofrobotrunners.org/resources/No-Man-Sky.pdf.

Zhang, Y.; et al. 2025. Concurrent Planning and Execution in Lifelong Multi-Agent Path Finding with Delay Probabilities. In *AAAI*, 23387–23394.

Zhong, X.; et al. 2022. Optimal and Bounded-Suboptimal Multi-Goal Task Assignment and Path Finding. In *ICRA*, 10731–10737.